

Meeting 6 - 14/08/2026

📅 Date	August 14, 2026
--------	-----------------

Architectural Design:

I split the energy functionality between **Python and Go** based on their existing responsibilities: Python already handles the satellite simulation and uses PyEphem for orbital and Sun-position calculations, so it calculates the Sun angle and utilization. Go already runs the NodeDaemon and manages runtime modules, so the `PowerModule` performs the actual power-generation and consumption calculations. **etcd acts as the interface between them**, passing `SolarInput` from Python to Go and `PowerStatus` back through the system

Python

Environmental/input calculation

```
Where is satellite?
  ↓
Where is Sun?
  ↓
What is Sun angle?
  ↓
What is utilization?
  ↓
SolarInput
```

Go

Power/physics calculation

```
SolarInput
+
Hardware specification
  ↓
Generation model
+
Consumption model
  ↓
PowerStatus
```

```
Python main.py
  |
  | WRITE
  ↓
etcd
  |
  | READ
  ↓
Go PowerModule
```

And then the reverse direction happens for the result:

```
Go PowerModule
  |
  | WRITE
  ↓
etcd
  |
  | READ
  ↓
other OpenSN components / API / console
```

`daemon/pkg/energy/model.go` (Go)

Purpose: The actual physics/math. Implements both equations from the slides as pure, independently-testable functions.

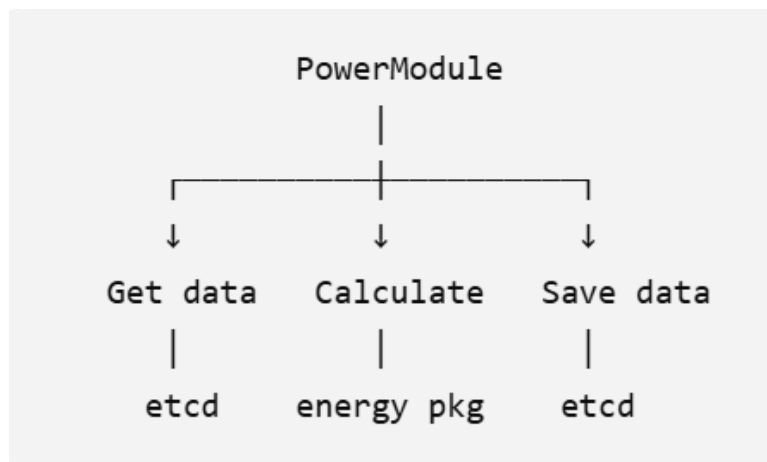
- `GenerationModel` / `ConsumptionModel` — strategy interfaces, so the math is swappable without touching calling code
- `ReflectivityLoss(gamma)` — the standard empirical reflectivity estimate ($\rho_0=0.04$, $n=5$)
- `FullPowerModel.Generate()` — implements $P = A \times \eta_{sc} \times B_0 \times \eta_{sys} \times \cos(\gamma) \times (1-p(\gamma))$, with the reflectivity function injectable

- `LinearConsumptionModel.Consume()` — implements $P_{idle} + (P_{max} - P_{idle}) \times u(t)$, with clamping for out-of-range utilization

`daemon/pkg/module/power_module.go` (Go)

Purpose: The actual running daemon module — `PowerModule`, built with the same `Base{sigChan, errChan, wg, daemonFunc, ModuleName}` shape as `MonitorModule` / `LinkModule`. Every tick (same interval as `monitor_interval`), it:

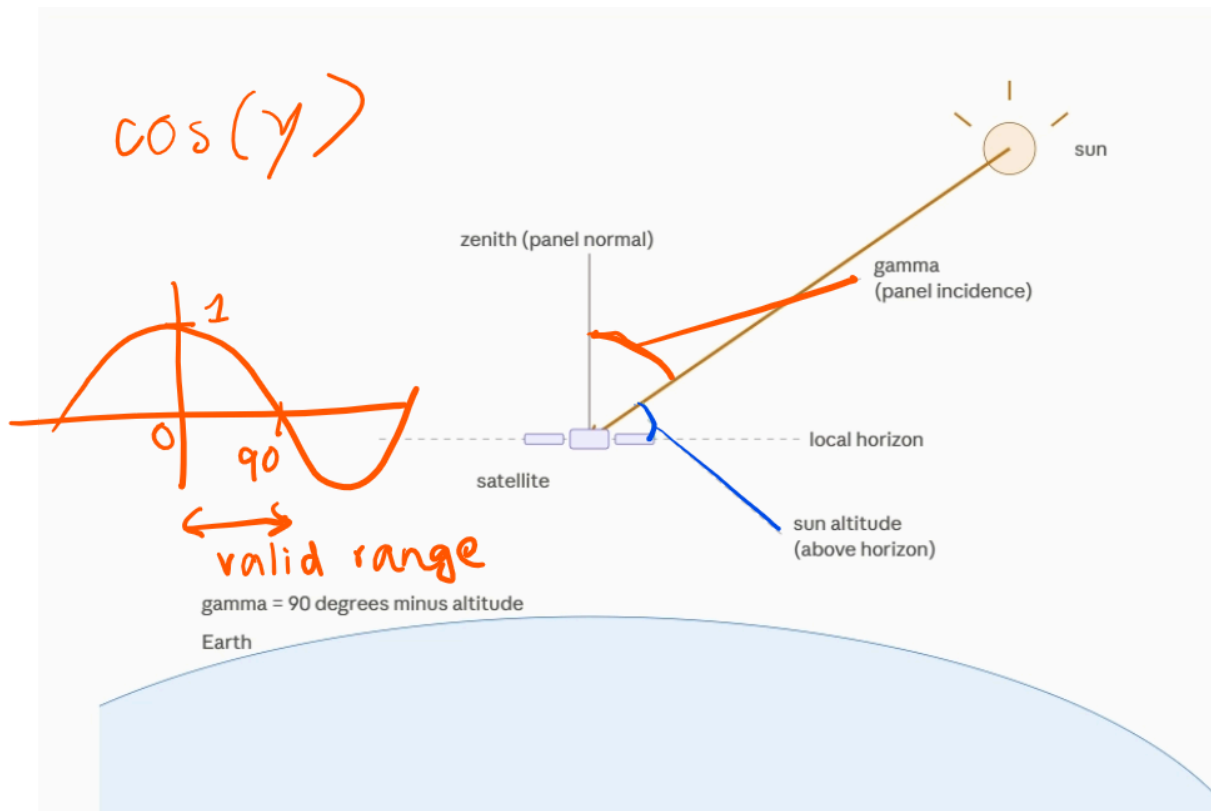
1. Reads the current instance list (satellite hardware specs, from `Instance.Extra`)
2. Reads all pushed `SolarInput` data
3. Computes generation + consumption per satellite
4. Writes `PowerStatus` back to etcd
Also defines the 5 `Ex*Key` constants (`panel_area_m2` etc.) and `specFromExtra()`, which parses+validates a satellite's hardware spec from its `Extra` map, silently skips ground stations).



`TopoConfigurators/Standard/opensn/synchronizer/sync_solar_input.py`

Python-side etcd read/write for `SolarInput`

`TopoConfigurators/Standard/trajectory.py`



TopoConfigurators/Standard/main.py

Added a small change to `main.py` so that, for each satellite, it calculates the **Sun angle and utilization** and writes them as `SolarInput` to etcd. This provides the Go `PowerModule` with the environmental inputs it needs to calculate power.

`get_utilization(instance_id)` This is not a proper function. Will have to use cgroups files to get actual utilisation. Right now, it just returns 0.5 for all satellites.

ISSUES I DISCOVERED

1. TopoConfigurators/Standard/main.py

FIXED!

there is a piece of code that **detect links that do not yet have an IPv4 address so it can assign one**, but it did not safely handle `None`.

```
if LINK_V4_ADDR_KEY not in link_info.address_infos[0] or \
    LINK_V4_ADDR_KEY not in link_info.address_infos[1]:
```

`address_infos[0]` or `[1]` can be `{}` or `None`. This check will throw an error `LINK_V4_ADDR_KEY not in None`

3. The `NodeDaemon` file relies on the `config/config.json` file in the daemon directory for configuration. That file is not in `.gitignore` and it's a tracked repo file. It contains **environment/runtime-specific settings**.

2.

The daemon Makefile does not build and copy the web UI into `share/static/dist/` before compiling the Go daemon, even though the Go code uses `go:embed` to package files from that directory. As a result, a fresh clone following the documented build process can produce a daemon without the embedded UI, causing the web console to return 404.

3.

OpenSN treats `NodeIndex` as a permanent identity for a machine, but `NodeDaemon` was incorrectly allocating a new index every time it restarted. The simulation data created under `/node_0/instances/...` stays there, while after a restart the daemon may identify itself as Node 1, 2, 3, etc. It then searches `/node_X/instances/...` using its new index and finds nothing.

Every time `NodeDaemon` starts, `allocNodeIndex()` runs and increments the counter, but instances are still sitting at `/node_0/`.

My computer is Node 0 but bug incorrectly assigns it Node 1.

Meeting Notes

CPU utilisation - consumption

add batteries at some point - find a battery model, charge, discharge, later more advanced versions-select it in config

ensure that we can disable energy module if needed